Artificial Intelligence

AI-2002

Lab 01

Introduction to AI and Python Programming

Course Code: AI-2002	Artificial Intelligence Lab

1. Objective 3
2. Introduction to AI 3
3. Implementation Platform for AI 3
3.1 History of Python 4
3.2 Distinct Features of Python 4
3.3 IDE for Python 5
4.1 Assigning Values to Variables 6
4.3 Basic Data Types 7
4.3 Conditional Statement 10
4.4 Loops 11
4.5 Functions 12
4.5.1 Types of Functions (Based on Arguments) 14
4.5.1 Types of Functions (Based on Number of Parameters) 14

1. Objective

1. Introduction to Artificial Intelligence, exposing students to different AI Problems and solutions.
2. To acquire programming skills in Python, Introduction to different Python Libraries for Artificial Intelligence.
3. Introduction to different IDE for python programming.
4. Solve some basic AI problem using the python programming language.

2. Introduction to AI

The term Artificial Intelligence was first coined decades ago in the year 1956 by John McCarthy at the Dartmouth conference. He defined AI as: "The science and engineering of making intelligent machines." In other words, Artificial Intelligence is the science of getting machines to think and make decisions like humans. In the recent past, AI has been able to accomplish this by creating machines and robots that have been used in a wide range of fields including healthcare, robotics, marketing, business analytics and many more.

3. Implementation Platform for AI

Artificial Intelligence has been around for over half a century now and its advancements are growing at an exponential rate. The demand for AI is at its peak and this lab course on Artificial Intelligence with Python will help you understand all the concepts of AI with practical implementations in Python. Python has made its way into the most complex technologies such as Artificial Intelligence, Machine Learning, Deep Learning, and so on. According to Guido van Rossum

"Python is an experiment in how much freedom programmers need. Too much freedom and nobody can read another's code; too little and expressive-nesses endangered."

Python is the top programming language in TIOBE and PYPL Index. According to PYPL, which publishes separate ranking for five countries, Python is the top language in all five countries (US, India, Germany, United Kingdom, and France). Python has taken a huge lead in these five countries.

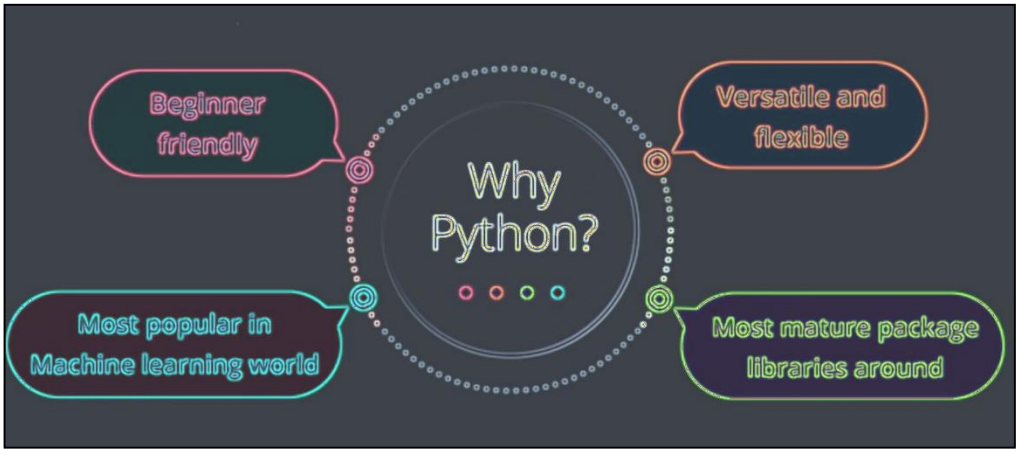
Jan 2023	Jan 2022	Change	Programming Language	Ratings	Change
1	1		Python	16.36%	+2.78%
2	2		C	16.26%	+3.82%
3	4	↑	C++	12.91%	+4.62%
4	3	↓	Java	12.21%	+1.55%
5	5		C#	5.73%	+0.05%
6	6		Visual Basic	4.64%	-0.10%
7	7		JavaScript	2.87%	+0.78%
8	9	↑	SQL	2.50%	+0.70%
9	8	↓	Assembly language	1.60%	-0.25%
10	11	↑	PHP	1.39%	-0.00%

3.1 History of Python

- Invented in the Netherlands, early 90s by Guido van Rossum
- Named after Monty Python
- Open sourced from the beginning
- Considered a scripting language, but is much more
- Scalable, object-oriented and functional from the beginning
- Used by Google from the beginning
- Increasingly popular

3.2 Distinct Features of Python

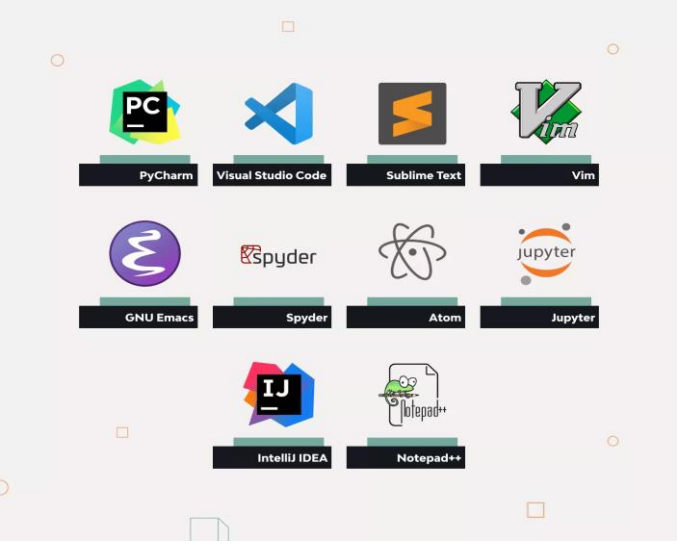
Python is a dynamic, high level, free open source and interpreted programming language. It supports object-oriented programming as well as procedural oriented programming. In Python, it's not necessary to declare the type of variable because it is a dynamically typed language.



- **Less Code:** Implementing AI involves tons of algorithms. Thanks to Pythons support for pre-defined packages, we don't have to code algorithms. And to make things easier, Python provides "check as you code" methodology that reduces the burden of testing the code.
- **Prebuilt Libraries**: Python has 100s of pre-built libraries to implement various Machine Learning and Deep Learning algorithms. So every time you want to run an algorithm on a dataset, all you have to do is install and load the necessary packages with a single command. Examples of pre-built libraries include NumPy, Kera's, TensorFlow, Pytorch, and so on.
- **Ease of learning:** Python uses a very simple syntax that can be used to implement simple computations like, the addition of two strings to complex processes such as building a Machine Learning model.
- **Platform Independent:** Python can run on multiple platforms including Windows, MacOS, Linux, Unix, and so on. While transferring code from one platform to the other you can make use of packages such as Py Installer that will take care of any dependency issues.
- **Massive Community Support:** Python has a huge community of users which is always helpful when we encounter coding errors. Apart from a huge fan following, Python has multiple communities, groups, and forums where programmers post their errors and help each other.

3.3 IDE for Python

An integrated development environment (IDE) is a software suite that consolidates basic tools required to write and test software.



Jupyter v/s PyCharm

	Features	Spyder	Jupyter Notebook	Google Colab
1	Installation	Yes	Yes	No
2	Launch	GUI	Web browser	Web browser
3	Internet Connection	No	No	Yes
4	Usage	Development	Data Science	Data Science
5	Visualisation	NA	Yes	Yes
6	Data	Local/Remote	Local/Remote	Google Drive
7	Power	CPU	CPU	CPU/GPU/TPU

4. Basic Python Syntax

```
print("Hello, World!")
```

4.1 Assigning Values to Variables

```
a = 2  # a is variable , we are assigning int value 2 to a using Equal to Operator (=)
b= 3.3  # b is variable , we are assigning float value 3.3 to a using Equal to Operator (=)
c= 'mehak'  # c is variable , we are assigning string value 'mehak 'to a using Equal to Operator (=)
```

User Input

```
name=input("enter your name")
print(name)
```

4.3 Basic Data Types

- ### Numeric Data Types

```
int_data = 17

float_data= 12.6

complex_data = 14 + 5j

print('Data : ', int_data , ', Type : ',type(int_data))

print('Data : ', float_data , ', Type : ',type(float_data))

print('Data : ', complex_data , ', Type : ',type(complex_data))
```

- ### Text Data Types

```
string_data = 'Machine Learning'
string_data_Alphabet = 'a'
string_paragraph = 'Machine learning is a subfield of artificial
intelligence that gives computers the ability to learn without
explicitly being programmed.'
string_num = '12'
print('Data : ', string_data , ', Type : ',type(string_data))
print('Data : ', string_data_Alphabet, ', Type :
',type(string_data_Alphabet))
print('Data : ', string_paragraph , ', Type :
',type(string_paragraph))
print('Data : ', string_num , ', Type : ',type(string_num))
```

Fetching Specific String or complete

```
t = 'Hello World!'

print (t) # Prints complete string

print (t[0])    # Prints first character of the string

print(t[2:5]) # Prints characters starting from 3rd to 5th

print (t[2:]) # Prints string starting from 3rd character

print (t* 2)  # Prints string two times

print (t + "TEST") # Prints concatenated string
```

- **Data Types to Store Collection**
- List
- Tuple
- Set
- Range
- Dictionary

```
list_data = [11,12,13,14,15, 'orange']

tuple_data = (12,15,17 , 'kiran')

set_data = {'ali', 'hassan', 'ayesha' , 12 ,18.6}

range_data = range(1,11)

print('List Data : ', list_data)

print('Tuple Data : ',tuple_data)

print('Set Data : ',set_data)

print('Range Data : ',range_data)
```

- **List Data Types**

```
list1 = ['physics', 'chemistry', 1997, 2000] # we can store any
type of data in List
list2 = [1, 2, 3, 4, 5, 6, 7 ]
print ("list1[0]: ", list1[0]) # Fetching item from index 0
print ("list2[1:5]: ", list2[1:5]) # Fetching item from index 1 to
4
list = ['physics', 'chemistry', 1997, 2000]
print("Value available at index 2 : ", list[2])

list[2] = 2001
print("New value available at index 2 : ", list[2])
# del item


list = ['physics', 'chemistry', 1997, 2000]
print(list)
del list[2]
print ("After deleting value at index 2 : ", list)
```

- **Tuple Data Types**

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )

tinytuple = (123, 'john')

print(tuple)  # Prints complete tuple

print(tuple[0])   # Prints first element of the tuple

print(tuple[1:3]) # Prints elements starting from 2nd till 3rd

print(tuple[2:])    # Prints elements starting from 3rd element

print(tinytuple * 2)  # Prints tuple two times print (tuple +
tinytuple) # Prints concatenated tuple

#we can update list but not tuple
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2  )
```

```
list = ['abcd', 786 , 2.23, 'john', 70.2 ]

tuple[2] = 1000 # Invalid syntax with tuple

list[2] = 1000  # Valid syntax with list
```

- **Dictionary Data Types**

```
dict = {}

dict['col1'] = "This is col one"

dict[2]= "This is col two"

print('=== Created Dictionary ==== ',dict)

tinydict = {'name': 'john','code':6734, 'dept': 'sales'}

print(dict['col1']) # Prints value for 'one' key

print(dict[2])  # Prints value for 2 key

print(tinydict) # Prints complete dictionary

print(tinydict.keys())  # Prints all the keys

print(tinydict.values()) # Prints all the values
```

4.3 Conditional Statement

If statement

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

If Else statement

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
else:
  print("You are not eligible to vote.")
```

Multiple Conditions

```
amount = int(input("Enter amount: "))
if amount<1000:

  discount = amount*0.05
  print("Discount",discount)

elif amount<5000:

  discount = amount*0.10
  print("Discount",discount)

else:

  discount= amount*0.15
  print ("Discount",discount)
  print ("Net payable:",amount-discount)
```

4.4 Loops

For Loop

```
for var in list(range(5)):
  print(var)
#other examples

for letter in 'Python': # traversal of a string sequence
  print ('Current Letter :', letter)

fruits = ['banana', 'apple', 'mango']
```

```
for fruit in fruits:  # traversal of List sequence
   print ('Current fruit :', fruit)

print ("Good bye!")
```

While Loop

```
count = 0
while (count < 9):
  print('The count is:', count)
  count = count + 1
  print('Good Bye')
```

4.5 Functions

Non-Parametric Function

```
# Function definition is here
def non_paraFunc ():
  print("This is Non_parametric Function")

non_paraFunc()
```

Parametric Function

```
# Function definition is here
def paraFunc (a,b):
  """This is Non_parametric Function"""
  #Takes the value of a and b parameters provided by the function
user
  return a + b

paraFunc(1,2)
```

Return Statement

```
# Function definition is here
def sum( arg1, arg2 ):
  # Add both the parameters and return them."
  total = arg1 + arg2
  print ("Inside the function : ", total)
  return total

# Now you can call sum function
total = sum( 10, 20 )
print ("Outside the function : ", total )

# Not returning

def print_message(message):
    print('Print Statement : ', message)
    return  # This is equivalent to return None

result = print_message("Hello, World!")
#We cannot store data if nothing is returned from the statement
print(result)  # Output: None
```

Return Statement

```
# Function definition is here
def sum( arg1, arg2 ):
  # Add both the parameters and return them."
  total = arg1 + arg2
  print ("Inside the function : ", total)
  return total

# Now you can call sum function
total = sum( 10, 20 )
print ("Outside the function : ", total )

# Not returning

def print_message(message):
    print('Print Statement : ', message)
    return  # This is equivalent to return None
```

```
result = print_message("Hello, World!")
#We cannot store data if nothing is returned from the statement
print(result)  # Output: None
```

4.5.1 Types of Functions (Based on Arguments)

- Positional
- Default
- Keyword

```
#Positional Argument

def details(name,age):
  print('Name : ', name)
  print('Age : ', age)

details(77,'Mehak')

#Default Argument

def details(name,age=13):
  print('Name : ', name)
  print('Age : ', age)

details('Mehak')

#Keyword  Argument

def details(name,age,height):
  print('Names : ' ,name)
  print('age : ' ,age)
  print('height : ' ,height)

details(name= 'Ali', age= 99, height= 5.5)
```

4.5.2 Types of Functions (Based on Number of Parameters)

- Fixed Length
- Variable Length

Fixed Length Parameter

```
#Function with Fixed Parameters
def add(a, b):
  '''A and B are Fixed Parameters'''
  return a + b

add(1,2,3) #If we provide extra arguments, It will return
TypeError
```

Variable Length Parameter

```
# Variable Length Args Argument

def details(*numbers):
  print('numbers : ' ,numbers)

details(1,2,3)

#Keyword Variable Length Args Argument

def details(**name):
  print('Names : ' ,name)

details(n1= 'Mehak', n2= 'Ali', n3= 'Zainab')
```

LAB TASKS